ИННОВАЦИИ В ФИНТЕХЕ:
ТЕОРИЯ И ПРАКТИКА

SAST и DAST

Как работают сканеры безопасности и что они видят в вашем проекте?

Сергей
Гречнев



НСПК | МИР | СБП

Знакомство со спикером



Сергей Гречнев

Начальник отдела тестирования безопасности приложений (AppSec)

> Проверки должны быть надёжными, а процессы эффективными

Общая структура лекции

01 Введение: зачем вообще нужны сканеры безопасности приложений?



Общая структура лекции

01 Введение: зачем вообще нужны сканеры безопасности приложений?

02 SAST: статический анализ кода, его особенности и применяемые техники



Общая структура лекции

01 Введение: зачем вообще нужны сканеры безопасности приложений?

02 SAST: статический анализ кода, его особенности и применяемые техники

03 DAST: динамический анализ кода, его особенности и методы решения сложностей



Общая структура лекции

01 Введение: зачем вообще нужны сканеры безопасности приложений?

02 SAST: статический анализ кода, его особенности и применяемые техники

03 DAST: динамический анализ кода, его особенности и методы решения сложностей

04 SAST и DAST: разница подходов



Общая структура лекции

01 Введение: зачем вообще нужны сканеры безопасности приложений?

02 SAST: статический анализ кода, его особенности и применяемые техники

03 DAST: динамический анализ кода, его особенности и методы решения сложностей

04 SAST и DAST: разница подходов

05 Развитие SAST и DAST



раздел

01

Введение



Зачем вообще нужны сканеры безопасности приложений?



Зачем вообще нужны сканеры безопасности приложений?

Экспоненциальный рост затрат на исправление ошибки



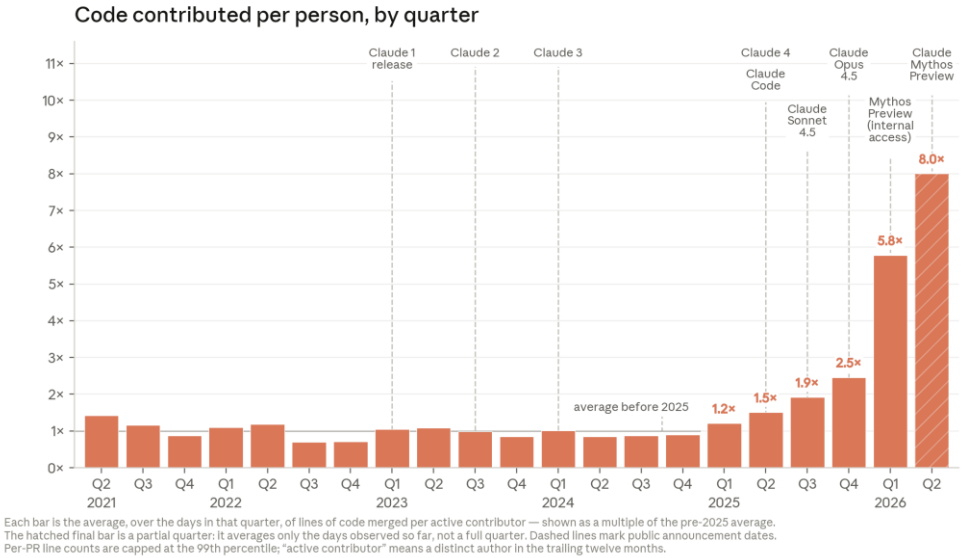


Зачем вообще нужны сканеры безопасности приложений?

Экспоненциальный рост затрат на исправление ошибки



Рост скорости разработки

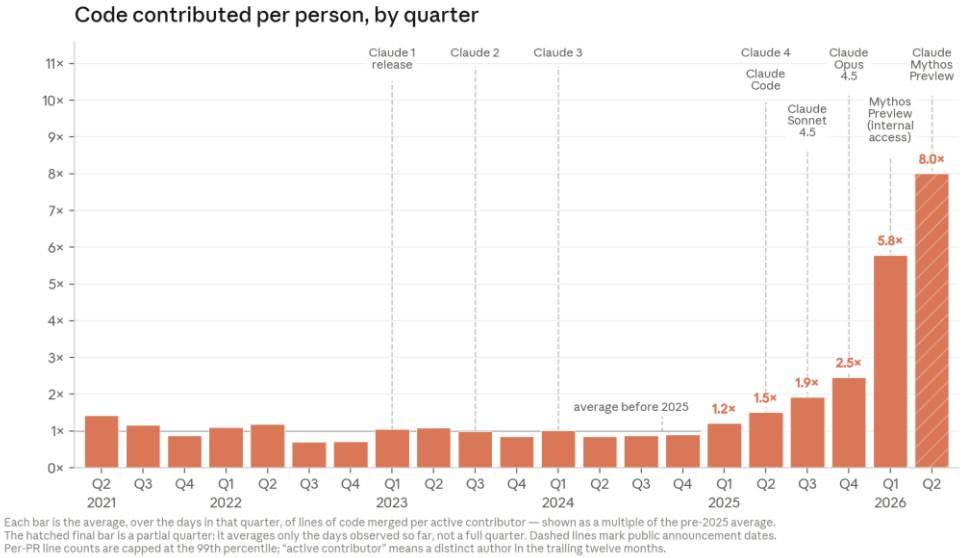


Зачем вообще нужны сканеры безопасности приложений?

Экспоненциальный рост затрат на исправление ошибки



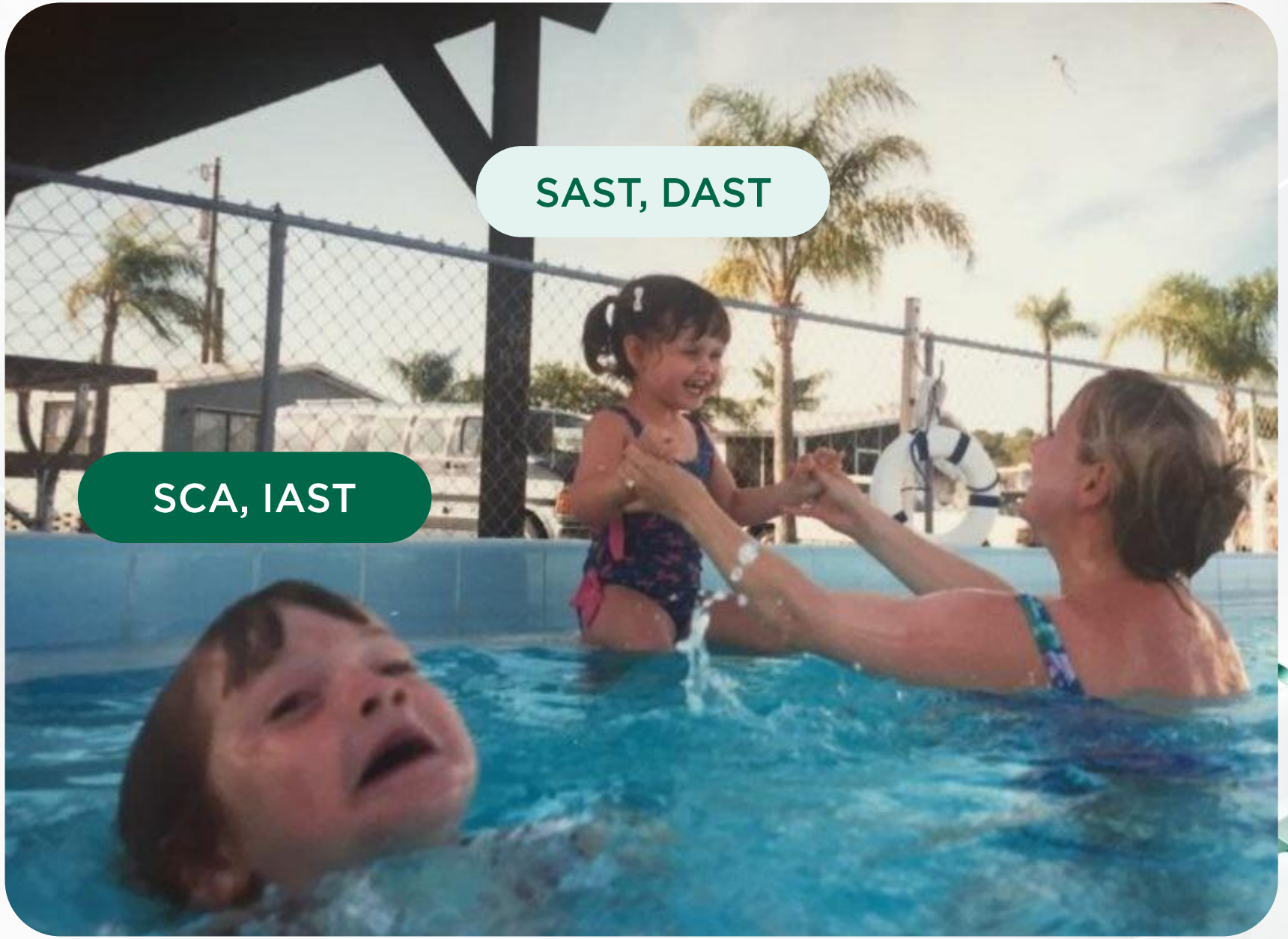
Рост скорости разработки



Решение: автоматизация рутины

SAST

DAST


SAST, DAST
SCA, IAST

раздел

02

SAST



Общее про SAST

Характеристики SAST

01 с доступом к исходному коду (whitebox)



Общее про SAST

Характеристики SAST

01 с доступом к исходному коду (whitebox)

02 без запуска



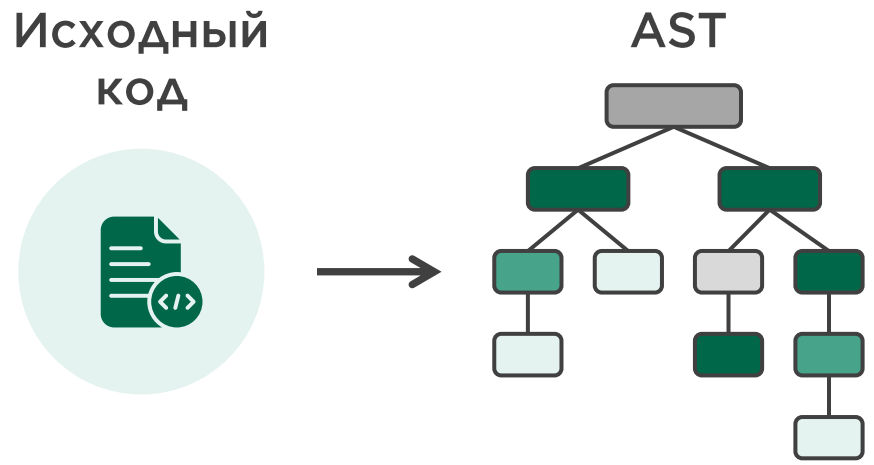
Общее про SAST

Характеристики SAST

01 с доступом к исходному коду (whitebox)

02 без запуска

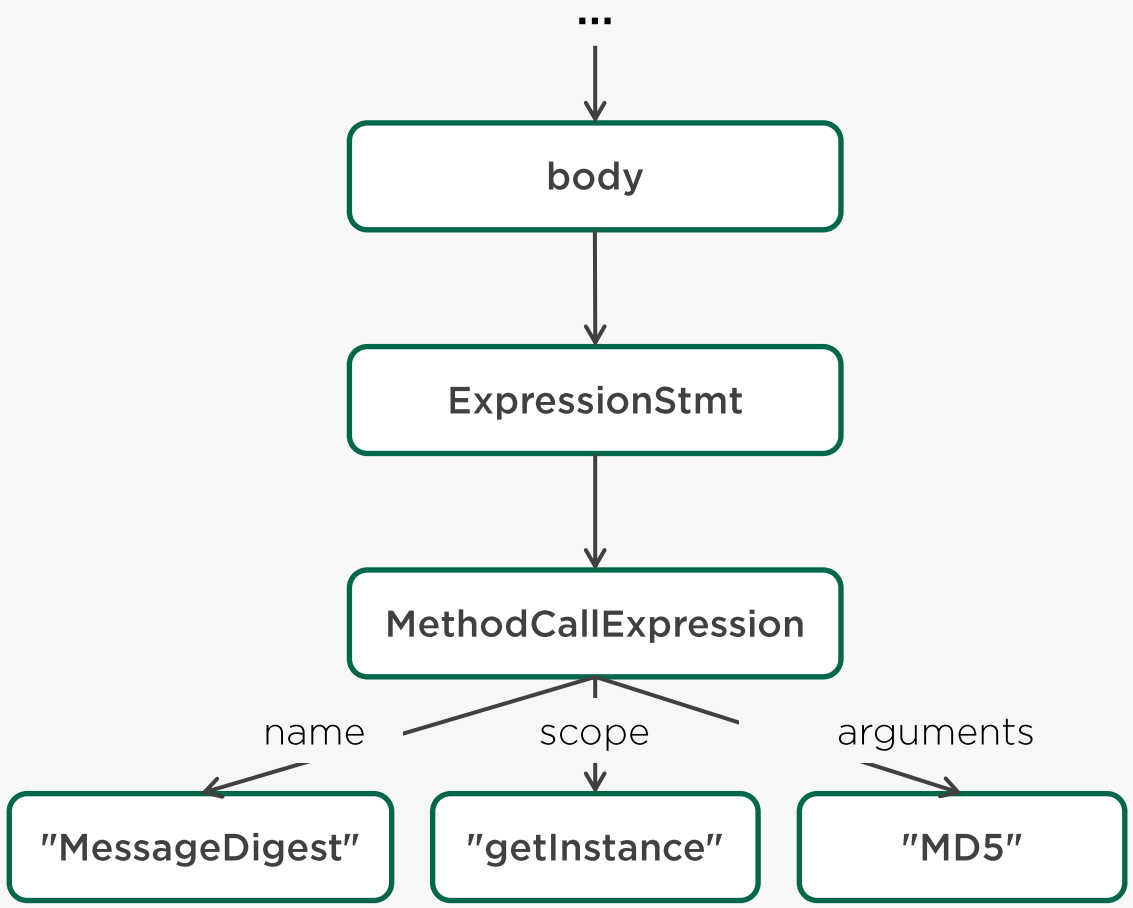
03 на основе абстрактного синтаксического AST



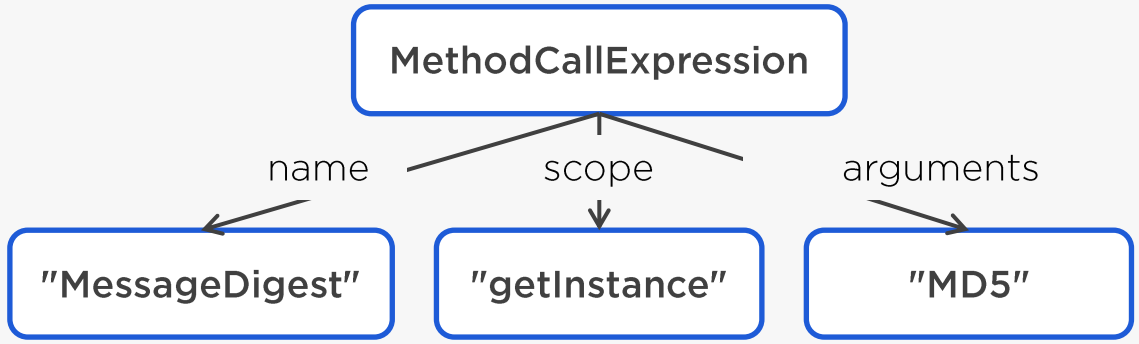


1. Pattern-matching

AST анализируемого кода

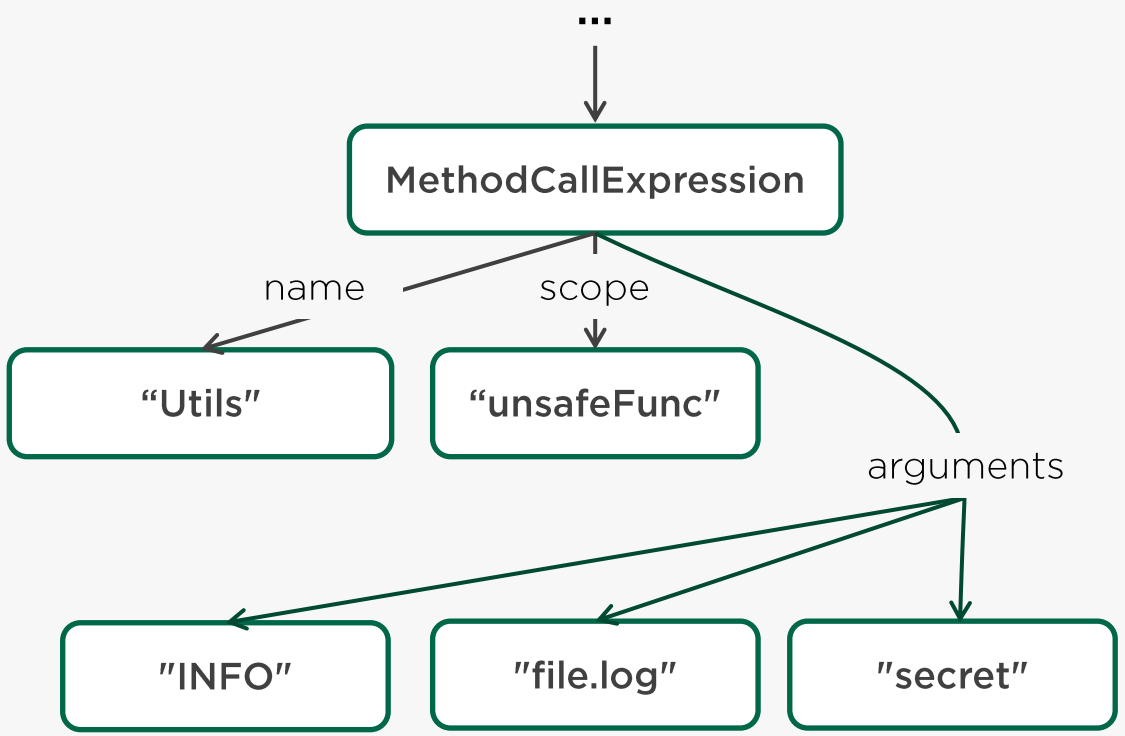


AST-шаблон правила анализа

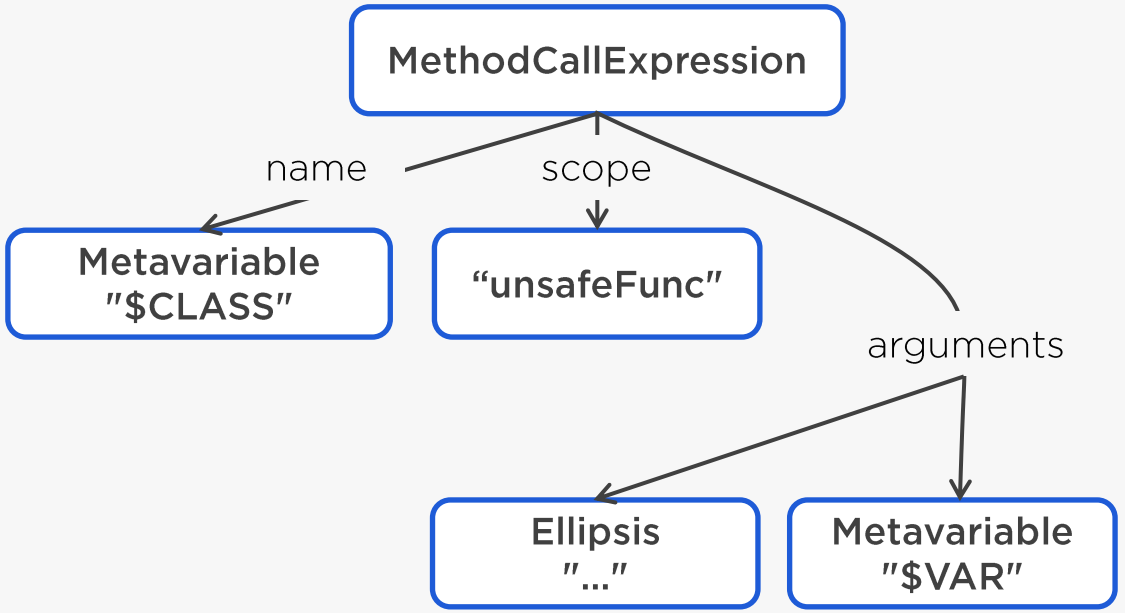


1. Pattern-matching

AST анализируемого кода



AST-шаблон правила анализа



1. Pattern-matching

Какие уязвимости обнаруживает

- использование устаревших и небезопасных функций (md5, sha1, java.util.Random, ...)
- hardcoded-secrets
- ошибки конфигурации безопасности в коде (csrf.disable(), allow_origins=['*'], requests.get(url, verify=False))

2. Taint-анализ. Что делает?

Анализ потока «заражённых» данных



2. Taint-анализ. Что делает?

Анализ потока «заражённых» данных

Источник:

точка ввода недоверенных данных (сетевые запросы, пользовательский ввод, БД, файлы, ...)



2. Taint-анализ. Что делает?

Анализ потока «заражённых» данных

Источник:

точка ввода недоверенных данных (сетевые запросы, пользовательский ввод, БД, файлы, ...)

Приемник:

функция, потенциально опасная при попадании недоверенных данных (запросы в БД, десериализация и парсинг, ...)



2. Taint-анализ. Что делает?

Анализ потока «заражённых» данных

Источник:

точка ввода недоверенных данных (сетевые запросы, пользовательский ввод, БД, файлы, ...)

Приемник:

функция, потенциально опасная при попадании недоверенных данных (запросы в БД, десериализация и парсинг, ...)

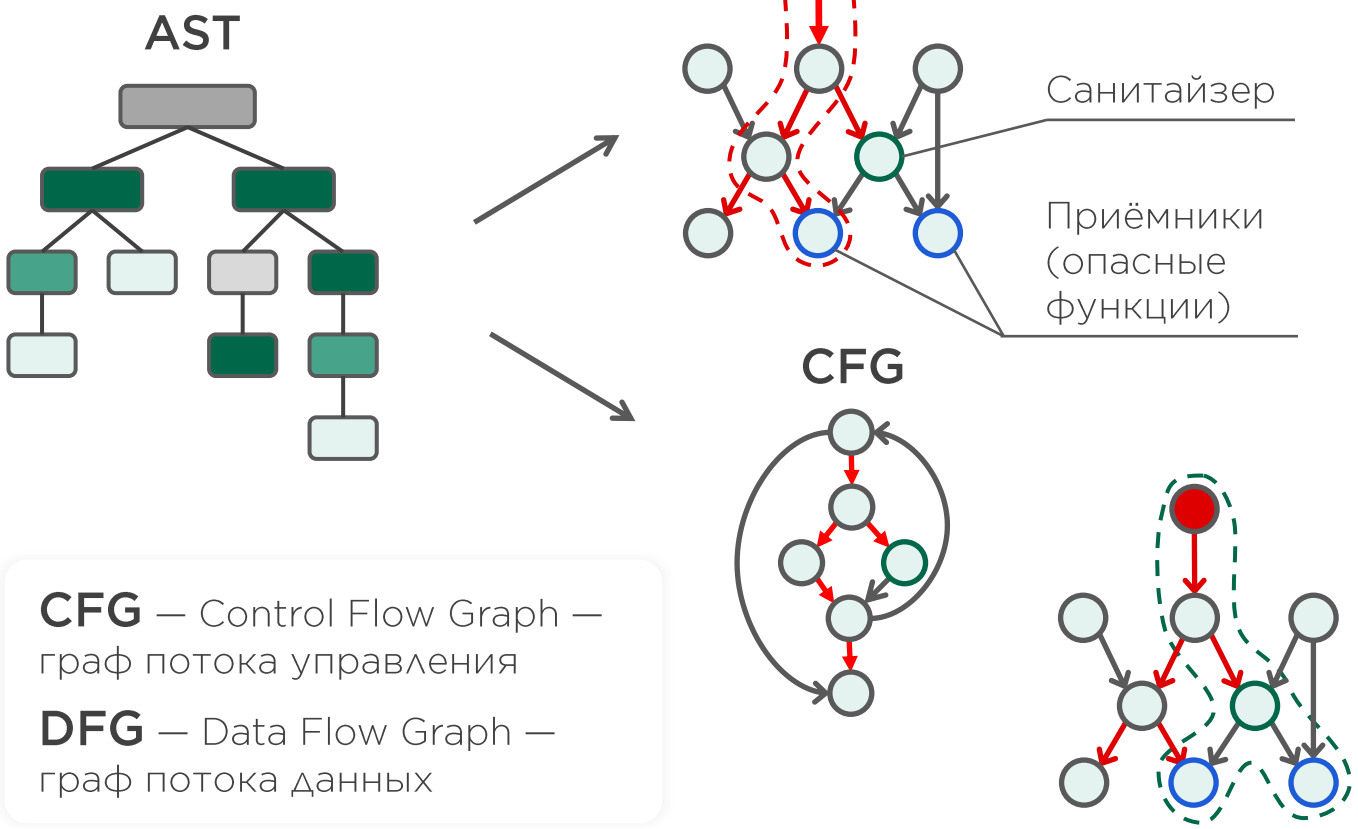
Санитайзер:

функция очистки данных. После прохождения санитайзера данные считаются безопасными для заданного контекста (SQL, HTML, ...)



2. Taint-анализ. Как делает?

Из AST строятся CFG и DFG



CFG — Control Flow Graph — граф потока управления

DFG — Data Flow Graph — граф потока данных

Анализ CFG позволяет:

- учесть порядок выполнения инструкций при анализе потоков данных
- чувствительность к путям
  (разделяет граф выполнения на две ветки. В одной ветке (где условие истинно) данные пойдут в приёмник очищенными, а во второй — загрязненными)
- выполнить межпроцедурный анализ
  (когда функция f() вызывает функцию g(), CFG строит ребро управления от точки вызова к объявлению функции g(), а затем возвращает его обратно (Return Edge). Это позволяет Taint-анализу точно знать, в какой именно контекст вызывающей функции должны вернуться «грязные» данные после выполнения метода)

2. Taint-анализ. Какие уязвимости обнаруживает

01 **Инъекции (подмешивание пользовательских данных в управляющие конструкции):**

- SQL injection
- XSS
- Command Injection
- Open Redirect
- Path Traversal

02 **Неконтролируемое потребление ресурсов**

- Buffer Overflow (если размер буфера определяется пользователем)
- Неконтролируемый размер цикла

03 **Уязвимости, возникающие в результате влияния недоверенных данных на критические функции**



3-4. Символьное исполнение и абстрактная интерпретация (краем уха)

Символьное исполнение

- программные переменные заменяются символьными переменными с максимальной областью значений
- условия и функции накладывают ограничения на область значений в отдельных частях программы. SMT-решатель определяет, можно ли подобрать такие входные данные, которые приведут программу к критической ошибке или нарушению безопасности

```
int x, y = read();

if ( y > 5) {          // [y>5]
    x = y;             // [x>5]
} else {               // [y<=5]
    x = 6L-y;          // [x>=1]
}

print(1.0/x);              // [x>=1]
```

Абстрактная интерпретация

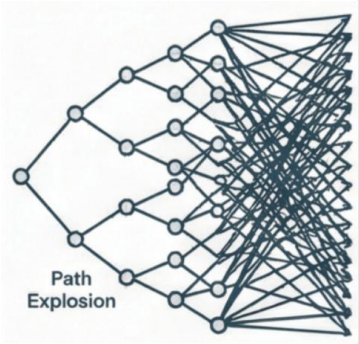
- переменные заменяются абстракциями: арифметический знак, принадлежность интервалу, иные свойства
- функции заменяются преобразованиями абстракций. Например, для вычисления знака произведения (-1)*33*(-7) можно выбрать абстракцию арифметического знака: (-)*(+)*(-) = (+)
- вывод определяемых свойств

3-4. Символьное исполнение и абстрактная интерпретация (краем уха)

Символьное исполнение

Помогает обнаружить:

- ошибки работы с памятью (Buffer Overflow/Underflow, Null Pointer Dereference, Use After Free)
- арифметические уязвимости (Integer Overflow/Underflow, Division by Zero)
- обход механизмов аутентификации и логические уязвимости
- все виды инъекций
- неконтролируемое потребление ресурсов
- **НО!** Проблема «Взрыва путей»



Абстрактная интерпретация

Помогает обнаружить:

- buffer Overflow
- out-of-bounds Read/Write
- integer Overflow/Underflow
- ошибки управления памятью и указателями
- инъекции, taint-анализ
- неконтролируемый размер цикла

SAST: резюме

- \+ Может работать с неполным и незапускающимся кодом, может включаться на ранних стадиях разработки (IDE, pre-commit hook, сборка на CI/CD)
- \+ Точное обнаружение уязвимости в коде (строка, фрагмент кода)
- +/− Время работы зависит от используемых техник: pattern-matching быстрый, symbolic execution очень медленный
- +/− Много ложных срабатываний
(чем проще техника, тем больше ложных срабатываний)



раздел 03

DAST



Общее про DAST

Тестируем развёрнутый сервис в модели blackbox: ничего не знаем о внутренностях сервиса

Этапы анализа:

01 Разведка

- определение доступных портов
- построение карты сайта

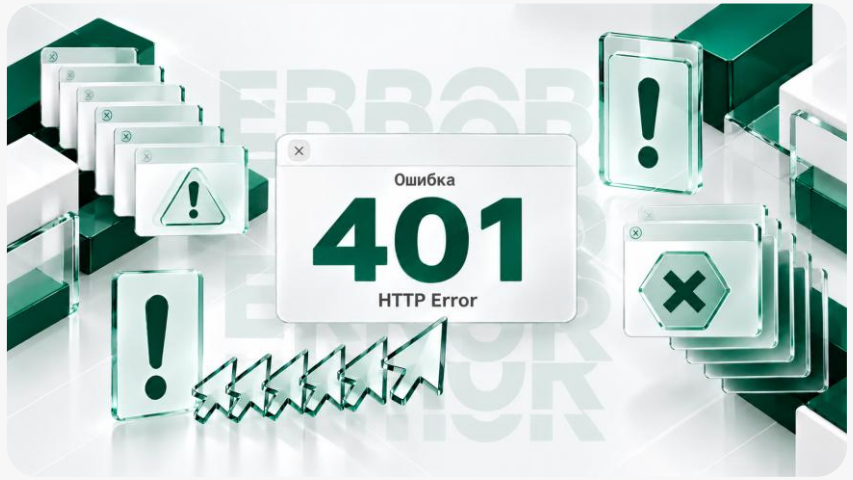
02 Атака

- отправка тестирующих запросов
- анализ ответов на наличие уязвимостей



Что видит DAST в вашем проекте?

- Ошибки аутентификации и сессий
- Время обработки запроса (DoS)
- Неправильно настроенные HTTP-заголовки безопасности (CORS, X-Frame-Options, CSP, STS)
- Отражённые инъекции
- Отклонения от ролевой модели (IDOR)





Проблема: DAST «из коробки» и строгая валидация

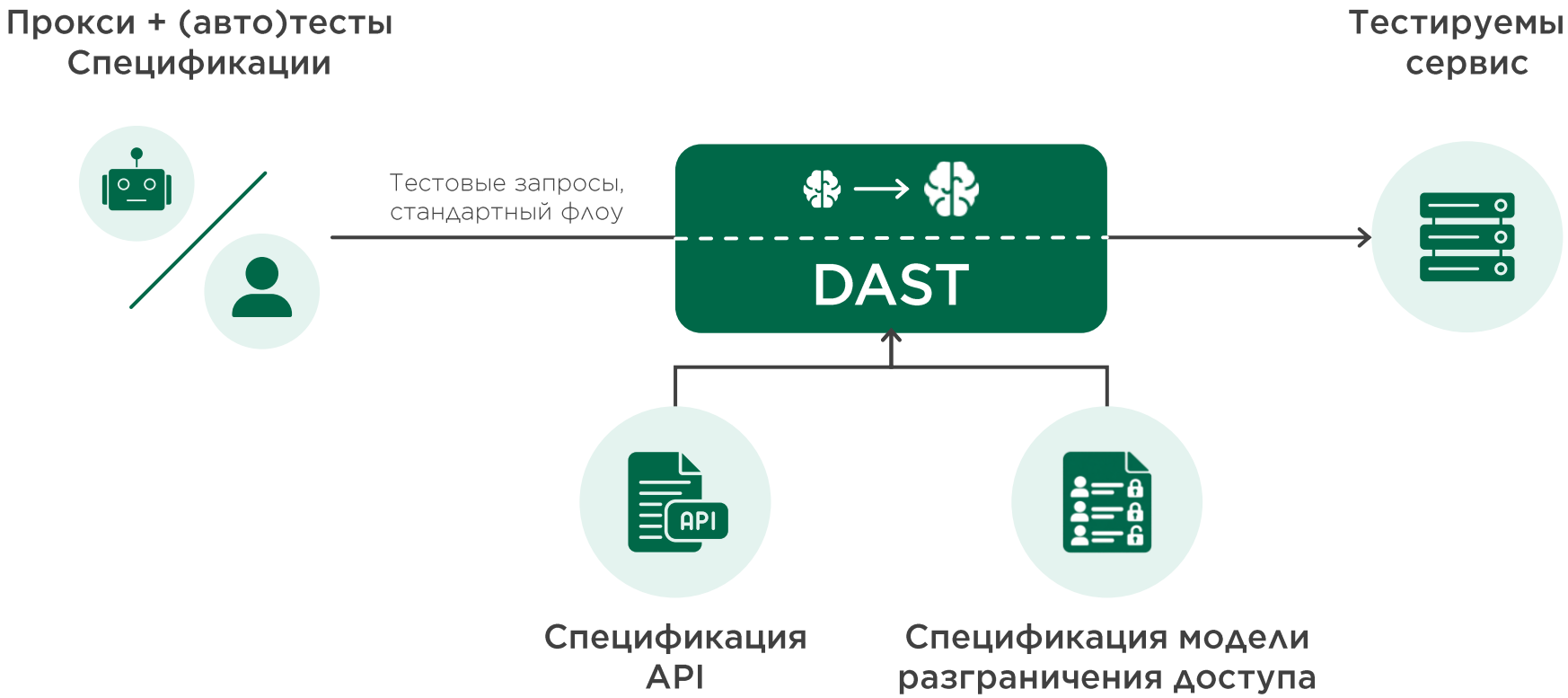
На сервере могут быть реализованы проверки корректности входных данных. Например, для номера банковской карты проверяться алгоритм Луна, а для адрес э/п соответствие шаблону

В модели blackbox внутренние правила валидации недоступны

Уязвимость может остаться не обнаруженной, когда данные не прошли этап валидации

Решение

Обучение

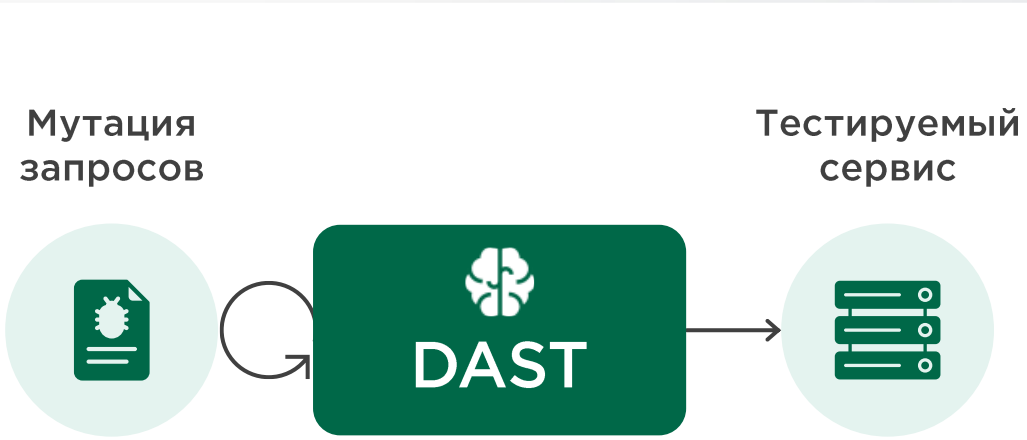


Решение

Атака

```
<order>
 <itemId>42</itemId>
 <promoCode>
  WELCOME2026
 </promoCode>
</order>
```

```
<order>
 <itemId>42</itemId>
 <promoCode>
  WELCOME2026' OR 1=1 --
 </promoCode>
</order>
```



DAST: резюме

+	Ниже доля ложных срабатываний (по сравнению с SAST)
−	Приложение уже должно запускаться, DAST работает только на поздних этапах разработки
−	Нет локализации недостатка в исходном коде, только параметры запроса и ответа
−	В среднем медленнее, чем SAST



раздел 04

SAST и DAST



НСПК | МИР | СБП

Сравнение подходов

Критерий сравнения	SAST	DAST
Метод	Белый ящик (полный доступ к исходному коду)	Чёрный ящик (без доступа к коду)
Этапы разработки	**Ранний** До первого коммита (IDE)	**Поздний** После успешной сборки и установки на тестовый стенд
Доля ложных срабатываний	**Высокая**	**Низкая**
Локализация ошибки	Фрагмент исходного кода	Параметры запроса и ответа
На что нацелен?	Ошибки логики кода, секреты	Проблемы рантайма, конфигурации

SAST+DAST





Объединение подходов

01 Встраиваем раздельно на разных этапах разработки

- SAST на этапе кодирования (легковесные IDE linter, pre-commit/-push hooks, CI)
- DAST на ночные сборки (так как он работает долго)



Объединение подходов

01 **Встраиваем раздельно на разных этапах разработки**

- SAST на этапе кодирования (легковесные IDE linter, pre-commit/-push hooks, CI)
- DAST на ночные сборки (так как он работает долго)

02 **Последовательное объединение SAST -> DAST:**

- SAST ищет потенциальные уязвимости в коде
- DAST проверяет гипотезы отправляя запросы и анализируя ответы



раздел 05

Развитие SAST и DAST



НСПК | МИР | СБП

Куда развивается [SD]AST?



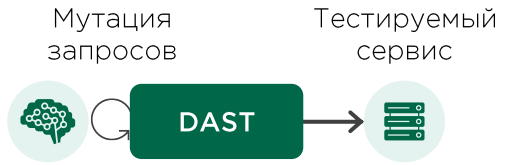
Что даёт AI?

01 SAST

- при использовании LLM совместно с другими SAST сканерами: анализ контекста срабатываний => обогащение данных, предварительный триаж, предложение исправлений кода
- самостоятельный анализ кода: анализ логических ошибок: LLM анализируют названия функций, переменных, комментарии и выводит назначение кода, его смысл

Что даёт AI?

02 DAST



- более адаптивная мутация запросов: не просто фиксированный список пэйлоадов, а анализ ответа сервиса и выбор стратегии атаки
- глубокий анализ ответов для определения наличия уязвимости
- агентный подход к тестированию (у каждого агента свои задачи, свои доступные инструменты):

Агент-Разведчик **Агент-Стратег** **Агент-Атакующий** **Агент-Валидатор**

Ограничения AI

01 Галлюцинации (гибридный подход, типовые методы ограничения галлюцинаций)

02 Ограничение размера контекстного окна (инкремент, только необходимое)

03 Нагрузка на AI (SLM, кеш)



раздел 05

Итоги



НСПК | МИР | СБП

Итоги

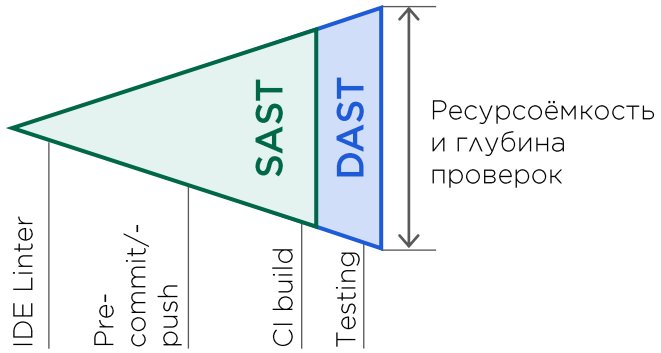
01



Безопасность — это процесс, а не разовое событие

Защищенный продукт создается не финальным аудитом перед релизом, а непрерывными проверками на каждом этапе разработки (DevSecOps)

02



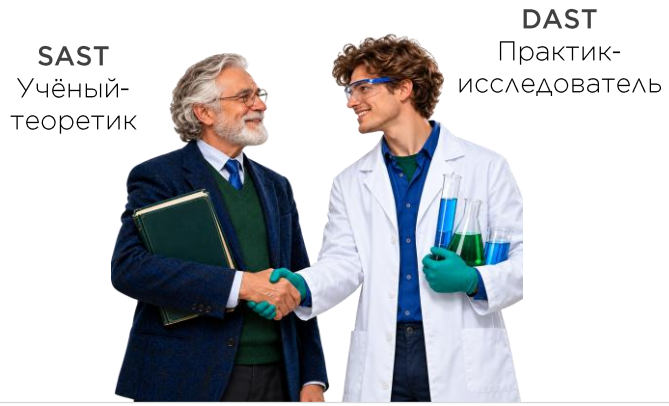
Каждому этапу — свой инструмент

Используйте SAST на ранних этапах:

- pattern-matching для поиска простых ошибок и секретов в коде с помощью
- taint-анализ для поиска инъекций
- символьное исполнение и абстрактную интерпретацию для поиска сложных логических уязвимостей

Ищите проблемы конфигурации и логики с помощью DAST на работающем стенде

03



Совместная работа сканеров экономит время

Связка двух инструментов («SAST находит, DAST проверяет») автоматически отсеивает ложные срабатывания и подсвечивает только реальные угрозы

Итоги

04



ИИ — ваш умный ассистент, а не замена

Он повышает адаптивность инструментов анализа и позволяет предварительно обработать результаты сканирования, обогащает данные, автоматизирует рутину, предлагая готовые исправления (auto-remediation)

05



Безопасность — это общая ответственность

Защищенность продукта создается не отдельным подразделением ИБ, а каждым участником команды на своем этапе. Безопасный код начинается с разработчика

SAST, DAST, LLM, да и люди могут допускать ошибки. Проверяйте важную информацию!

ИННОВАЦИИ В ФИНТЕХЕ:
ТЕОРИЯ И ПРАКТИКА

Спасибо
за внимание!

Сергей
Гречнев



НСПК | МИР | СБП